

Programmation Graphique Avancée dans Godot

💡 Objectifs du document

Ce document est un complément au cours de programmation graphique (420-J30-BB). Les sessions 1 à 12 couvrent le pipeline de rendu avec Babylon.js et Three.js. Ici, on plonge **sous le capot** de Godot — un moteur qui expose le pipeline graphique d’une manière différente, plus proche du métal, avec son propre langage de shader (GDShader) et son éditeur visuel (VisualShader). On va couvrir les détails avancés que les autres moteurs cachent derrière leurs abstractions : les trois passes d’un spatial shader, les varyings, le rendu PBR, les modes de mélange, et les pièges classiques.

💡 Prérequis

- **Sessions 3-4** : vous savez ce qu’est un vertex shader et un fragment shader, et vous avez écrit du GLSL.
- **Session 5** : vous comprenez l’éclairage direct (Lambert, Blinn-Phong) et les propriétés de surface (albedo, roughness, metallic).
- **Session 8** : vous savez ce qu’est un render target.
- **Session 11** : vous comprenez le tone mapping et le gamma correction.

Partie 1 : Le Pipeline de Rendu de Godot

1. Les modes de rendu

Forward+ vs Mobile vs Clustered

Godot 4 propose trois modes de rendu :

- **Forward+** (par défaut) — le mode desktop. Utilise le rendu forward avec un pré-pass de clustering pour les lumières. Supporte les ombres, le PBR complet, le HDR, le post-processing. C’est l’équivalent du pipeline qu’on a vu avec Three.js, mais optimisé.
- **Mobile** — optimisé pour mobile et web. Moins de lumières dynamiques, pas de clustering, simplifications du PBR.
- **Clustered** — un hybride qui découpe l’écran en clusters (cellules 2D) pour limiter les lumières par fragment. Plus proche d’un deferred rendering.

💡 Pourquoi ça nous intéresse

Le mode de rendu affecte **combien de lumières** vous pouvez avoir. En Forward+, Godot découpe l’écran en clusters (une grille 2D) et pour chaque cluster, ne garde que les lumières qui l’affectent. C’est la même idée que le frustum culling (Session 12), mais appliquée aux lumières : on ne calcule l’éclairage que pour les lumières qui touchent un cluster donné. C’est ce qui permet d’avoir des dizaines de lumières dynamiques sans tuer le framerate.

2. Le pipeline d’un frame“

Les passes d'un frame Godot

1. **Depth pre-pass** (optionnel) — rend la profondeur de la scène dans un depth buffer, sans couleur. Accélère le rendu en éliminant les fragments cachés avant les calculs d’éclairage.

2. **Shadow maps** — pour chaque lumière qui projette des ombres, rend la scène depuis la lumière (comme la Session 10).
3. **Géométrie + G-Buffer** — rend la scène, calcule l'éclairage par fragment. En Forward+, chaque fragment reçoit la liste des lumières de son cluster.
4. **Post-processing** — tone mapping, bloom, SSAO, etc. (Session 11).
5. **Composition finale** — affichage à l'écran avec gamma correction.

Partie 2 : Les Trois Passes d'un Spatial Shader

🗨️ La structure d'un shader spatial

Un `shader_type spatial`; dans Godot peut contenir **trois** fonctions, chacune à un endroit précis du pipeline :

```
shader_type spatial;
```

```
void vertex() {  
    // Manipule les vertices (position, normal, UV)  
}  
  
void fragment() {  
    // Définit les propriétés de surface (albedo, roughness, metallic)  
}  
  
void light() {  
    // Modifie comment chaque lumière affecte cette surface  
}
```

Ces trois fonctions ne sont **pas** indépendantes — elles communiquent via des variables partagées.

1. La fonction `vertex()`

Rôle

S'exécute **une fois par vertex**. Reçoit les attributs du mesh (position, normale, UV, couleur) et peut les modifier. C'est ici qu'on fait :

- **Displacement** — déplacer les vertices (vagues sur l'eau, vent dans l'herbe).
- **Vertex animation** — animer la géométrie au niveau du vertex (plus efficace qu'au niveau du fragment).
- **Skinning / morph** — déformer selon les bones ou les blend shapes.

Ondulation de drapeau.

```
void vertex() {  
    // Déplace les vertices en X selon le temps et la hauteur  
    float wave = sin(TIME + VERTEX.y * 4.0) * 0.1;  
    VERTEX.x += wave * UV.y; // UV.y = 0 en bas (fixe), 1 en haut (mobile)  
}
```

🗨️ Les entrées et sorties

Entrées (lecture seule) :

- VERTEX — position en **local space** (avant transformation).

- NORMAL — normale en local space.
- UV, UV2 — coordonnées de texture.
- COLOR — couleur du vertex (si le mesh en a une).
- TIME — temps écoulé (secondes).

Sorties (écriture) :

- VERTEX — position modifiée (sera transformée en world/view space par Godot).
- NORMAL — normale modifiée.
- UV, UV2 — UV modifiés.

2. La fonction fragment()

Rôle

S'exécute **une fois par pixel** (fragment). Définit les propriétés de la surface — ce que le pixel "est". C'est l'équivalent du fragment shader qu'on a écrit en Session 3-4, mais avec des sorties structurées pour le PBR.

Les sorties principales

- ALBEDO (vec3) — la couleur de base de la surface, **avant** éclairage. C'est la "peinture".
- ROUGHNESS (float) — 0 = miroir, 1 = mat. Contrôle la taille du reflet spéculaire.
- METALLIC (float) — 0 = diélectrique (bois, plastique), 1 = métal. Change complètement la réflexion.
- NORMAL (vec3) — la normale de surface (peut être modifiée par une normal map).
- EMISSION (vec3) — couleur émise (indépendante de l'éclairage). Pour les lampes, le néon, la lave.
- ALPHA (float) — transparence (0 = invisible, 1 = opaque). Nécessite render_mode blend_mix.
- ALPHA_SCISSOR (float) — cutoff alpha (comme alphaTest en Three.js, Session 12).

Texture + normal map.

```
uniform sampler2D albedo_tex;
uniform sampler2D normal_tex;

void fragment() {
    ALBEDO = texture(albedo_tex, UV).rgb;
    // Normal map : convertir de [0,1] vers [-1,1]
    NORMAL_MAP = texture(normal_tex, UV).rgb;
    NORMAL_MAP_DEPTH = 1.0; // intensité de la perturbation
    ROUGHNESS = 0.5;
    METALLIC = 0.0;
}
```

3. La fonction light()

Rôle

S'exécute **une fois par pixel, par lumière**. C'est la passe qui calcule **comment une lumière affecte la surface**. Godot fait le PBR par défaut — la fonction light() vous permet de **modifier** ce calcul.

! Ce n'est PAS un varying

ALBEDO n'est pas un varying entre `fragment()` et `light()`. Les deux fonctions s'exécutent **dans le même fragment shader** — Godot les compile en un seul shader GLSL. La structure interne ressemble à :

```
struct Surface {
    vec3 albedo;
    float roughness;
    float metallic;
    vec3 normal;
    vec3 emission;
};

void fragment(in vec2 uv, out Surface s) {
    s.albedo = ...; // votre code
    s.roughness = ...;
}

void light(in Surface s, in LightData light, out vec3 contribution) {
    // s.albedo est lisible — c'est ce que fragment() a mis
    contribution = light.color * dot(s.normal, light.dir) * s.albedo;
}

void main() {
    Surface s;
    fragment(uv, s); // 1 fois

    vec3 total = vec3(0.0);
    for (int i = 0; i < num_lights; i++) {
        vec3 c;
        light(s, lights[i], c); // N fois
        total += c;
    }
    gl_FragColor = vec4(s.albedo * total + s.emission, 1.0);
}
```

Donc ALBEDO est un **membre de struct** passé d'une fonction à l'autre — pas d'interpolation, pas de varying. Les deux fonctions tournent sur le même pixel.

Les entrées de `light()`

Lecture seule :

- LIGHT (vec3) — couleur et intensité de la lumière **après atténuation**.
- LIGHT_COLOR (vec3) — couleur brute de la lumière (sans atténuation).
- ATTENUATION (float) — atténuation (distance, angle pour les spots).
- ALBEDO (vec3) — ce que `fragment()` a mis (lecture seule).
- ROUGHNESS, METALLIC — de `fragment()`.
- NORMAL (vec3) — normale de surface.
- VIEW (vec3) — direction du pixel vers la caméra.
- SPECULAR (vec3) — la réflexion spéculaire calculée par Godot.

Sortie :

- LIGHT (vec3) — vous pouvez le **modifier** pour changer la contribution de cette lumière.

- SPECULAR (vec3) — modifier le spéculaire.
- ALBEDO (vec3) — oui, modifiable ici aussi (rare).

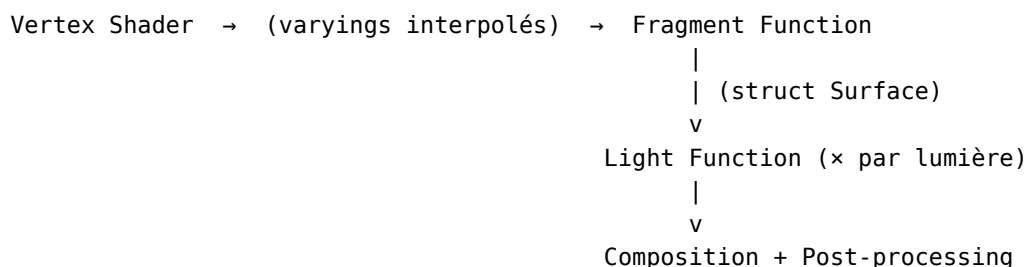
💡 Quand utiliser light() ?

La plupart du temps, **on ne l'écrit pas** — Godot fait le PBR correctement. On l'écrit pour :

- **Toon shading** — quantifier la lumière en steps (pas de dégradé).
- **Rim light** — ajouter un halo sur les bords ($\text{dot}(\text{NORMAL}, \text{VIEW})$ faible).
- **Subsurface scattering fake** — laisser la lumière “traverser” les surfaces fines.
- **Atténuation custom** — forme de cône de lumière non standard.

4. Le flux de données complet

Schéma du pipeline



- **Vertex → Fragment** : les varyings (UV, NORMAL world, VERTEX world) sont **interpolés** à travers les triangles. C’est là que le mot “varying” a du sens.
- **Fragment → Light** : passage de struct, **même pixel**, pas d’interpolation. La fonction `light()` tourne plusieurs fois mais `fragment()` une seule.

Partie 3 : Les Varyings — Ce Qui Voyage des Vertices aux Pixels

1. Qu’est-ce qu’un varying ?

Définition

Un **varying** est une variable passée du vertex shader au fragment shader. Sa valeur est **interpolée** à travers chaque triangle, en fonction de la position du fragment.

Si un vertex A met $UV = (0, 0)$ et un vertex B met $UV = (1, 1)$, un fragment au milieu du triangle recevra $UV = (0.5, 0.5)$. C’est cette interpolation qui permet aux textures d’être appliquées correctement : chaque pixel reçoit l’UV interpolé, et échantillonne la texture à cet endroit.

Visualisation de l'interpolation.

```

// Vertex shader
void vertex() {
    COLOR = vec4(UV, 0.0, 1.0); // COLOR est un varying
}

// Fragment shader
void fragment() {
    ALBEDO = COLOR.rgb; // reçoit l'UV interpolé
}
  
```

Le résultat : un dégradé de couleur à travers le triangle. En bas-gauche (UV 0,0) = noir, en haut-droite (UV 1,1) = jaune (R=1, G=1, B=0).

2. Les varyings implicites de Godot

Ce que Godot fournit automatiquement

Godot crée des varyings pour vous :

- UV — coordonnées de texture du mesh.
- NORMAL — normale en **view space** (si non modifiée).
- VERTEX — position en **view space** (si non modifiée).
- COLOR — couleur du vertex (si le mesh en a une).
- BINORMAL, TANGENT — pour le normal mapping.

Vous pouvez aussi créer vos propres varyings :

```
varying vec3 my_custom_data;

void vertex() {
    my_custom_data = vec3(1.0, 0.0, 0.0);
}

void fragment() {
    ALBEDO = my_custom_data; // interpolé à travers le triangle
}
```

3. Le piège de l'interpolation perspective

⚠ L'interpolation n'est pas linéaire

L'interpolation des varyings n'est pas linéaire en distance — elle est **perspective-correct**. Un fragment plus proche de la caméra “pèse” plus qu'un fragment lointain. C'est pourquoi une texture ne se déforme pas bizarrement quand on regarde un triangle de biais : sans correction perspective, la texture semblerait “courber” près des bords.

En GLSL moderne (et dans Godot), cette correction est automatique. Mais si vous passez des données non-géométriques (ex. un ID d'objet), l'interpolation peut produire des valeurs intermédiaires sans sens — il faut alors utiliser des **flat varyings** (pas d'interpolation) ou des **integer varyings**.

```
flat varying int object_id; // pas d'interpolation
```

Partie 4 : Le PBR dans Godot

1. Le modèle de Cook-Torrance

Rappel du PBR (Session 5)

Le PBR (Physically Based Rendering) repose sur le modèle de Cook-Torrance :

$$f_{r(\omega_i, \omega_o)} = k_d \text{ Lambert} + k_s \text{ Cook-Torrance}$$

- k_d — composante diffuse (la lumière qui pénètre la surface et ressort aléatoirement).
- k_s — composante spéculaire (la lumière qui rebondit directement, comme un miroir).
- ALBEDO contrôle la couleur diffuse.

- ROUGHNESS contrôle la rugosité du microfacet (taille du reflet spéculaire).
- METALLIC contrôle le ratio $\frac{k_d}{k_s}$: un métal n'a pas de diffuse, que du spéculaire teinté.

2. Les propriétés de surface dans Godot

Tableau récapitulatif

Propriété	Type	Plage	Effet
ALBEDO	vec3	[0, 1]	Couleur de base, avant éclairage
ROUGHNESS	float	[0, 1]	0 = miroir, 1 = mat
METALLIC	float	[0, 1]	0 = diélectrique, 1 = métal
NORMAL_MAP	vec3	[0, 1]	Perturbe la normale (normal mapping)
NORMAL_MAP_DEPTH	float	[0, +∞]	Intensité de la perturbation
EMISSION	vec3	[0, +∞]	Lumière émise (indépendante de l'éclairage)
ALPHA	float	[0, 1]	Transparence (blend mode)
ALPHA_SCISSOR	float	[0, 1]	Cutoff alpha (1 bit, pas de tri)
RIM	float	[0, 1]	Intensité du rim light
RIM_TINT	float	[0, 1]	Teinte du rim
CLEARCOAT	float	[0, 1]	Vernis (couche transparente brillante)
CLEARCOAT_ROUGHNESS	float	[0, 1]	Rugosité du vernis
ANISOTROPY	float	[-1, 1]	Anisotropie (reflets allongés, cheveux)
SUBSURFACE_SCATTERING	float	[0, 1]	Diffusion sous la surface (peau)
BACKLIGHT	float	[0, 1]	Rétroéclairage (feuilles, papier)

3. Le metallic et le diélectrique

💡 Pourquoi metallic est binaire en pratique

En théorie, METALLIC est un float [0, 1]. En pratique, les matériaux réels sont **soit** métalliques (1.0) **soit** diélectriques (0.0) — les valeurs intermédiaires n'existent pas dans la nature. Les artistes utilisent 0 ou 1, rarement entre. Une valeur de 0.5 donne un rendu “plastique métallisé” qui n'a pas de correspondance physique.

Exception : la **rouille** sur un métal — c'est de l'oxyde, qui est diélectrique. Donc une texture métallique rouillée aura METALLIC = 0 sur la rouille et METALLIC = 1 sur le métal nu. C'est pour ça qu'on utilise une **texture** de metallic, pas une valeur constante.

Partie 5 : Les Modes de Rendu (render_mode)

1. Blend modes“

Les modes de mélange

```
render_mode blend_mix;      // alpha blend standard (transparent)
render_mode blend_add;      // addition (feu, explosions, néon)
render_mode blend_sub;      // soustraction (rare)
render_mode blend_mul;      // multiplication (verre teinté)
```

- **blend_mix** — l’objet est transparent et se mélange avec le fond. Nécessite un tri par distance (les objets transparents ne s’écrivent pas dans le depth buffer).
- **blend_add** — la couleur de l’objet **s’additionne** au fond. Plus l’objet est lumineux, plus il “blanchit” le fond. Utilisé pour les particules de feu, les lasers, les halos.
- **blend_mul** — la couleur **multiplie** le fond. Un objet rouge assombrit tout sauf le rouge. Utilisé pour le verre teinté, les gel colorés.

2. Cull modes

Quelles faces on rend

```
render_mode cull_back;      // ne rend que les faces avant (défaut)
render_mode cull_front;     // ne rend que les faces arrière
render_mode cull_disabled;  // rend les deux (double-sided)
```

Rappel Session 1 : le **backface culling** élimine les triangles dont la normale pointe away de la caméra. `cull_back` (défaut) garde les faces avant. Pour une feuille de papier ou un drapeau, on veut `cull_disabled` — on voit les deux côtés.

3. Depth modes

Écriture et test de profondeur

```
render_mode depth_draw_opaque; // écrit dans le depth buffer (défaut)
render_mode depth_draw_always; // toujours écrire
render_mode depth_draw_never;  // jamais écrire (transparent)
render_mode depth_test_disabled; // ne pas tester (toujours dessiner)
```

Les objets **opaques** écrivent dans le depth buffer. Les objets **transparentes** ne le font pas — sinon ils masqueraient les objets derrière. C’est pour ça que les transparents doivent être triés par distance.

4. Unshaded et wireframe

Modes spéciaux

```
render_mode unshaded;      // ignore l'éclairage (émission seulement)
render_mode wireframe;     // rend en wireframe
```

- **unshaded** — l’objet n’est pas affecté par les lumières. Seul `ALBEDO` et `EMISSION` sont visibles. Utilisé pour les HUD, les indicateurs, les objets qui doivent toujours être visibles.
- **wireframe** — rend les edges du mesh. Utile pour le debug.

Partie 6 : Les Uniforms — Passer des Données au Shader

Définition

Un **uniform** est une variable passée du CPU au GPU, **constante pour tous les vertices/fragments** d'un draw call. C'est comment on contrôle un shader depuis le code (GDScript) ou l'inspecteur.

```
uniform vec4 albedo_color : source_color = vec4(1.0, 0.0, 0.0, 1.0);
uniform float intensity : hint_range(0.0, 1.0) = 0.5;
uniform sampler2D main_texture;
uniform sampler2DArray texture_array;
uniform sampler3D volume_texture;
```

1. Les hints d'uniform

Hints pour l'inspecteur

Godot utilise des **hints** pour afficher le bon widget dans l'inspecteur :

- : `source_color` — affiche un color picker (avec gamma correct).
- : `hint_range(0, 1)` — affiche un slider.
- : `hint_range(0, 10, 0.1)` — slider avec pas de 0.1.
- : `hint_default_white` — texture blanche par défaut.
- : `hint_default_black` — texture noire par défaut.
- : `hint_normal` — indique que c'est une normal map (affiche l'aperçu bleu).
- : `hint_aniso` — texture d'anisotropie.

Sans hint, l'inspecteur affiche un champ générique.

2. Accéder aux uniforms depuis GDScript

Modifier un uniform en code.

```
# Récupérer le material
var mat = $MeshInstance3D.get_active_material()

# Modifier un uniform
mat.set_shader_parameter("albedo_color", Color(1.0, 0.0, 0.0, 1.0))
mat.set_shader_parameter("intensity", 0.8)

# Passer une texture
var tex = load("res://textures/wood.png")
mat.set_shader_parameter("main_texture", tex)
```

Partie 7 : Le VisualShader Editor

Le VisualShader

Le **VisualShader** est l'éditeur visuel de shader de Godot. Au lieu d'écrire du GDShader en texte, on connecte des nœuds dans un graphe. Le résultat est compilé en GDShader en interne — c'est exactement le même langage, juste une autre interface.

1. Les trois onglets

Vertex, Fragment, Light

L'éditeur a trois onglets, correspondant aux trois fonctions :

- **Vertex** — manipulation des vertices (déplacement, animation).
- **Fragment** — propriétés de surface (albedo, roughness, metallic, normal).
- **Light** — modification de l'éclairage (toon, rim, SSS).

Chaque onglet a son propre graphe et son propre nœud Output. Les nœuds d'un onglet ne peuvent pas se connecter aux nœuds d'un autre — la communication entre onglets se fait via les variables de sortie (ALBEDO, ROUGHNESS, etc.).

2. Le nœud Output

Le nœud de sortie

Chaque onglet a un nœud **Output** (toujours à droite, ID 0). Ses ports dépendent de l'onglet :

- **Fragment Output** — ALBEDO, ALPHA, ROUGHNESS, METALLIC, NORMAL, EMISSION, etc.
- **Light Output** — LIGHT, SPECULAR, ALBEDO (modifiable ici).
- **Vertex Output** — VERTEX, NORMAL, UV, etc.

Connecter un nœud à un port de l'Output = définir cette propriété. Si un port n'est pas connecté, la valeur par défaut est utilisée.

3. Les nœuds principaux

Catégories de nœuds

- **Input** — les entrées du shader (UV, NORMAL, TIME, SCREEN_UV, etc.). C'est l'équivalent des variables prédéfinies en GDShader.
- **Constant** — valeurs fixes (Color, Float, Vector, Boolean).
- **Operator** — opérations mathématiques (Add, Multiply, Mix, Dot, Cross, etc.).
- **Texture** — échantillonnage de textures (Texture2D, TextureCube, etc.).
- **Scalar** — opérations sur scalaires (sin, cos, pow, clamp, smoothstep).
- **Vector** — opérations sur vecteurs (normalize, length, distance).
- **Procedural** — textures procédurales (noise, voronoi, fractal).
- **Transform** — matrices de transformation.
- **Output** — le nœud de sortie (un par onglet).

4. Avantages et limites du VisualShader

Pour et contre

Avantages :

- Pas besoin de connaître la syntaxe GDShader.
- Visualisation immédiate des connexions.
- Bon pour les shaders simples à moyens.

Limites :

- UX peu polie (recherche de nœuds lente, ports petits).
- Difficile pour les shaders complexes (le graphe devient illisible).
- Pas de boucles, pas de fonctions personnalisées.

- Plus lent à itérer que du texte (plus de clics).
- Le code généré n'est pas toujours optimal.

Recommandation : apprendre le GDShader en texte d'abord, utiliser le VisualShader pour le prototypage rapide ou l'enseignement.

Partie 8 : Exemples Pratiques

1. Toon shader (cel shading)

Quantifier la lumière en steps.

```
shader_type spatial;

void fragment() {
    ALBEDO = vec3(0.8, 0.6, 0.2); // couleur de base
    ROUGHNESS = 1.0;
}

void light() {
    // Quantifier l'intensité en 3 niveaux
    float intensity = dot(NORMAL, LIGHT_DIRECTION);
    intensity = clamp(intensity, 0.0, 1.0);

    // 3 steps : 0.0, 0.5, 1.0
    float stepped = floor(intensity * 3.0) / 3.0;

    // Appliquer
    LIGHT = LIGHT_COLOR * stepped * ATTENUATION;
}
```

Le résultat : au lieu d'un dégradé smooth, on a 3 bandes de couleur. C'est le look anime/cartoon.

2. Rim light (halo sur les bords)

Backlight sur les silhouettes.

```
shader_type spatial;

void fragment() {
    ALBEDO = vec3(0.2, 0.4, 0.8);
}

void light() {
    // dot(NORMAL, VIEW) = 1 au centre, 0 au bord
    float rim = 1.0 - dot(NORMAL, VIEW);
    rim = pow(rim, 3.0); // sharper

    // Ajouter du rim à la lumière
    LIGHT += LIGHT_COLOR * rim * ATTENUATION;
}
```

Le résultat : un halo lumineux sur les bords de l'objet, visible quand la normale pointe away de la caméra.

3. Eau avec reflets et vagues

Shader d'eau simple.

```

shader_type spatial;

uniform sampler2D normal_map1;
uniform sampler2D normal_map2;
uniform float wave_speed = 0.05;

void vertex() {
    // Ondulation des vertices
    VERTEX.y += sin(TIME * 2.0 + VERTEX.x * 3.0) * 0.1;
    VERTEX.y += cos(TIME * 1.5 + VERTEX.z * 2.0) * 0.08;
}

void fragment() {
    // Combiner deux normal maps qui défilent
    vec2 uv_flow = UV + TIME * wave_speed;
    vec3 n1 = texture(normal_map1, uv_flow).rgb;
    vec3 n2 = texture(normal_map2, uv_flow * 2.0).rgb;
    NORMAL_MAP = mix(n1, n2, 0.5);
    NORMAL_MAP_DEPTH = 0.5;

    // Couleur de l'eau
    ALBEDO = vec3(0.0, 0.3, 0.5);
    ROUGHNESS = 0.1;    // presque miroir
    METALLIC = 0.0;
}

```

4. Dissolve effect

Dissoudre un objet avec une texture de bruit.

```

shader_type spatial;
render_mode blend_mix, depth_draw_opaque;

uniform sampler2D noise_texture;
uniform float dissolve_amount : hint_range(0.0, 1.0) = 0.5;
uniform vec4 edge_color : source_color = vec4(1.0, 0.5, 0.0, 1.0);

void fragment() {
    float noise = texture(noise_texture, UV).r;

    // Si le noise est en dessous du seuil, on est "dissous"
    if (noise < dissolve_amount) {
        discard; // jette le fragment
    }

    // Bordure lumineuse sur le seuil
    float edge = smoothstep(dissolve_amount, dissolve_amount + 0.05, noise);
    edge = 1.0 - edge;

    ALBEDO = vec3(0.5);
    EMISSION = edge_color.rgb * edge * 3.0;
}

```

Le résultat : l'objet se dissout progressivement, avec un bord lumineux (effet "burning").

Partie 9 : Les Pièges Classiques

1. Oublier le gamma correction

Le piège sRGB

Si vous lisez une couleur dans une texture et la passez à ALBEDO, Godot fait la conversion sRGB → linéaire automatiquement **si** la texture est marquée comme sRGB. Mais si vous faites des calculs sur la couleur (multiplication, mix), ils doivent se faire en **linéaire**.

Si vos couleurs paraissent “trop saturées” ou “trop sombres au milieu”, c’est probablement un problème de gamma. Vérifiez que vos textures sont importées en sRGB et que `render_mode` n’a pas `color_mode_linear` (qui désactive la conversion).

2. Normal map en tangent space

Le repère de la normal map

Les normal maps stockent les normales en **tangent space** — un repère local à chaque triangle. Godot fait la conversion vers le world/view space automatiquement quand vous utilisez `NORMAL_MAP`. Mais si vous calculez votre propre normale dans `fragment()`, elle doit être en **view space** (le repère de Godot par défaut).

Si votre éclairage semble “inverse” ou “bizarre” après une normal map, vérifiez :

- Que la texture est marquée comme **Normal Map** dans l’import (sinon Godot la traite comme sRGB).
- Que vous n’avez pas écrasé `NORMAL` après avoir mis `NORMAL_MAP`.

3. Transparence et tri

Pourquoi les transparents clignotent

Les objets transparents (`blend_mix + ALPHA < 1`) ne s’écrivent pas dans le depth buffer. Donc si deux transparents se chevauchent, l’ordre de rendu décide qui est devant. Si cet ordre change (la caméra bouge), les objets “clignotent”.

Solutions :

- **Alpha scissor** (`ALPHA_SCISSOR = 0.5`) — 1 bit, pas de tri, écrit dans le depth. Pour l’herbe, les feuilles.
- **Dither** (`render_mode alpha_antialiasing_edge`) — pseudo-transparency avec un pattern, pas de tri.
- **Manual sort** — trier les transparents par distance en GDScript (coûteux).

4. TIME et le deterministic shader

TIME n’est pas un varying

TIME est un **uniform** — il a la même valeur pour tous les fragments d’un draw call. Il change entre les frames, mais pas entre les pixels. C’est pour ça qu’on peut l’utiliser pour animer un shader : tous les pixels voient le même TIME, donc l’animation est cohérente.

Mais si vous voulez un bruit **différent** par pixel, il faut combiner TIME avec UV ou VERTEX :

```
float noise = fract(sin(dot(UV + TIME * 0.1, vec2(12.9898, 78.233)))) * 43758.5453);
```

Conclusion

Ce qu'on a vu

- **Pipeline Godot** : Forward+ avec clustering des lumières, depth pre-pass, shadow maps, post-processing.
- **Trois passes d'un spatial shader** : `vertex()` (vertices), `fragment()` (surface), `light()` (éclairage par lumière).
- **Varyings** : interpolation perspective-correct entre vertex et fragment. **Pas** entre fragment et light (même pixel, struct).
- **PBR** : Cook-Torrance avec albedo, roughness, metallic. Godot gère le calcul, on fournit les propriétés.
- **Render modes** : blend, cull, depth, unshaded, wireframe.
- **Uniforms** : passage CPU → GPU, avec hints pour l'inspecteur.
- **VisualShader** : éditeur visuel, trois onglets, nœud Output par onglet. Bon pour prototyper, limité pour les shaders complexes.
- **Exemples** : toon shader, rim light, eau, dissolve.
- **Pièges** : gamma correction, tangent space, transparence, TIME.

Godot expose le pipeline graphique d'une manière plus transparente que Babylon.js ou Three.js. Comprendre ces détails vous permet de diagnostiquer les problèmes et d'écrire des shaders que les autres moteurs ne vous laissent pas écrire.

Pour aller plus loin

- **Godot Shader Documentation** : docs.godotengine.org → **Shaders** → **Your first shader**.
- **ShaderToy** : pour expérimenter avec des fragment shaders en WebGL.
- **The Book of Shaders** : thebookofshaders.com — référence pour les shaders procéduraux.
- **LearnOpenGL** : learnopengl.com — pour comprendre le pipeline OpenGL sous-jacent.